Aufgabe: Vererbung (Inheritance)

Für: Michael **Referenz:** W3Schools – C# Inheritance **Bearbeitungszeit:** ca. 45–60 Min.
Voraussetzung: Diese Aufgabe baut bewusst auf dem auf, was du schon gelernt hast – Klassen/Objekte, Konstruktoren, Access Modifiers und Properties. Neu ist nur **Vererbung**.

---

1. Was du schon kannst (kurzer Rückblick)

Bevor es losgeht, kurz auffrischen – du brauchst das gleich wieder:

- **Klasse & Objekt:** Eine Klasse ist der Bauplan, ein Objekt ein konkretes Exemplar davon (`new Klasse()`)
- **Konstruktor:** Füllt ein neues Objekt beim Erzeugen mit sinnvollen Startwerten
- **Access Modifiers:** `private` (nur innerhalb der Klasse sichtbar) vs. `public` (von aussen zugänglich)
- **Property:** Kontrollierter Zugriff auf ein privates Feld über `get` / `set`

Wenn eines davon noch wackelig ist, kurz in den entsprechenden Unterlagen nachschauen, bevor du weitermachst.

2. Neu: Vererbung

Lies zuerst das W3Schools-Kapitel: **C# Inheritance**. Es zeigt dieses Beispiel:

```
class Vehicle  // Basisklasse (parent)
{
  public string brand = "Ford";
  public void honk()
  {
    Console.WriteLine("Tuut, tuut!");
  }
}

class Car : Vehicle  // abgeleitete Klasse (child)
{
  public string modelName = "Mustang";
}
```

Die Kernidee: `Car` erbt alles von `Vehicle` (Feld `brand`, Methode `honk()`) und ergänzt eigene Sachen (`modelName`). Der Doppelpunkt `:` bedeutet “erbt von”.

Begriffe: - **Basisklasse (parent):** die Klasse, von der geerbt wird (`Vehicle`) - **Abgeleitete Klasse (child):** die Klasse, die erbt (`Car`) - **`sealed`:** verhindert, dass von einer Klasse überhaupt geerbt werden darf

Verbindung zu dem, was du schon kennst: Vererbung ändert nichts an Konstruktoren, Access Modifiers oder Properties – sie kommen einfach mit. Genau das übst du jetzt.

---

3. Deine Aufgabe: Fahrzeug-Hierarchie

Du baust eine kleine Fahrzeugverwaltung. Anders als im W3Schools-Beispiel (öffentliche Felder) nutzt du dabei **Konstruktor, private Felder und Properties**, weil du das schon kannst und es in der Praxis der Normalfall ist.

Schritt 1 – Basisklasse `Fahrzeug` (Wiederholung)

Baue eine Klasse `Fahrzeug` mit: - privatem Feld `_marke` (string) - Property `Marke` (nur `get`, im Konstruktor gesetzt) - Konstruktor, der `Marke` setzt - einer öffentlichen Methode `Hupen()`, die `"Tuut, tuut!"` ausgibt

Das ist reine Wiederholung von Phase 2 – nichts Neues.

Schritt 2 – Abgeleitete Klasse `Auto`

```
class Auto : Fahrzeug
{
    // hier kommt dein Code hin
}
```

- Ergänze ein eigenes privates Feld `_modell` mit Property `Modell`
- Schreibe einen Konstruktor für `Auto`, der **zwei** Parameter annimmt (`marke`, `modell`) und `marke` über `base(marke)` an den `Fahrzeug`-Konstruktor weiterreicht (genau wie in Phase 3 – Session 1 besprochen)

Schritt 3 – Ausprobieren

Im `Main()`:

```
Auto meinAuto = new Auto("Ford", "Mustang");
meinAuto.Hupen();                    // von Fahrzeug geerbt
Console.WriteLine(meinAuto.Marke + " " + meinAuto.Modell);
```

Vorhersage zuerst: Was gibt die Konsole aus, bevor du startest? Schreib es auf, dann prüfen.

Schritt 4 – Zweite abgeleitete Klasse

Baue analog eine Klasse `Motorrad : Fahrzeug` mit einem eigenen Feld `Hubraum` (int). Erzeuge ein `Motorrad`-Objekt und ruf `Hupen()` sowie dein neues Feld auf.

Leitfrage: Was ist bei `Auto` und `Motorrad` identisch (weil geerbt), was ist unterschiedlich?

Schritt 5 – `sealed` ausprobieren (kurz)

Setz `sealed` vor `class Fahrzeug` und versuch zu kompilieren. Lies die Fehlermeldung.

Frage: Warum könnte man in der Praxis eine Klasse absichtlich versiegeln (`sealed`) wollen?

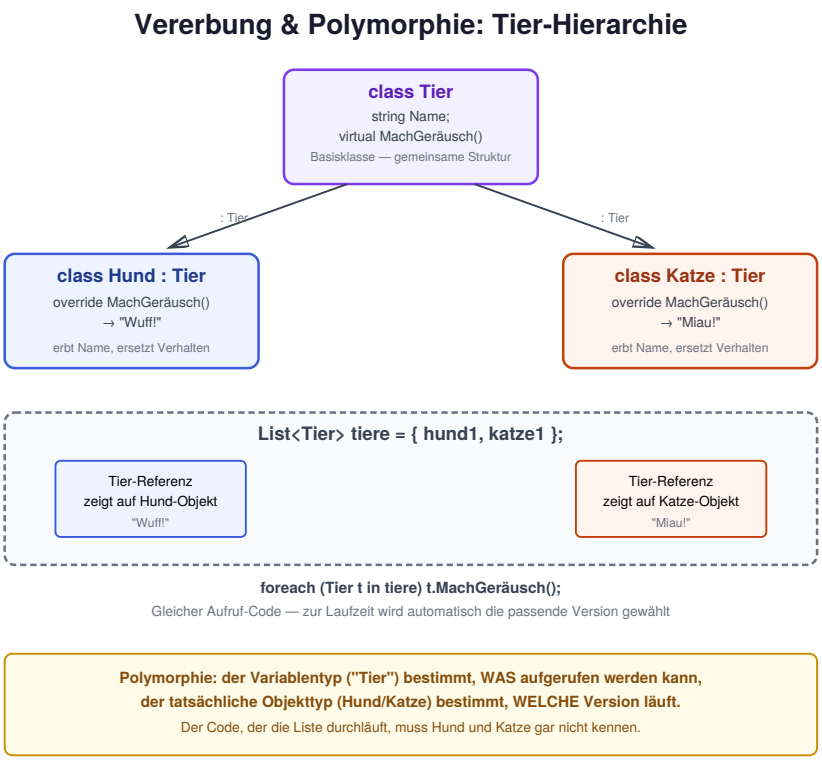
---

4. Stolperstein-Check (typische Fehler)

Bevor du fertig bist, prüf bewusst:

- ☐ Hast du `base(marke)` im `Auto`-Konstruktor nicht vergessen? (Ohne das gibt's einen Compilerfehler – lies ihn, falls er auftaucht.)
- ☐ Ist `_marke` wirklich `private`, aber trotzdem über `Marke` von `Auto` aus lesbar? (Warum funktioniert das, obwohl `_marke` privat ist? → Weil die *Property* `public` ist, nicht das Feld.)
- ☐ Kannst du auf ein `Auto`-Objekt sowohl geerbte als auch eigene Mitglieder aufrufen?

5. Bild zur Orientierung



Vererbung

Der obere Teil des Bildes (Basisklasse → abgeleitete Klassen) ist das, was du gerade übst. Der untere Teil (Polymorphie mit `List<Tier>`) ist das nächste Thema – heute nur zur Einordnung, noch nicht Teil dieser Aufgabe.

---

6. Abschluss-Fragen (mündlich beantworten)

1. Was bedeutet der Doppelpunkt `:` in `class Auto : Fahrzeug`?
2. Was übernimmt `Auto` automatisch von `Fahrzeug`, ohne dass du es erneut schreiben musstest?
3. Warum musstest du `base(marke)` in den `Auto`-Konstruktor schreiben?
4. Wofür ist `sealed` gut?

7. Weiterführend

Wenn das sitzt, schau dir zur Vertiefung das W3Schools-Kapitel **Polymorphism** an – das ist der nächste logische Schritt und baut direkt auf dieser Aufgabe auf.